

Introduction to Computing (CS 101)

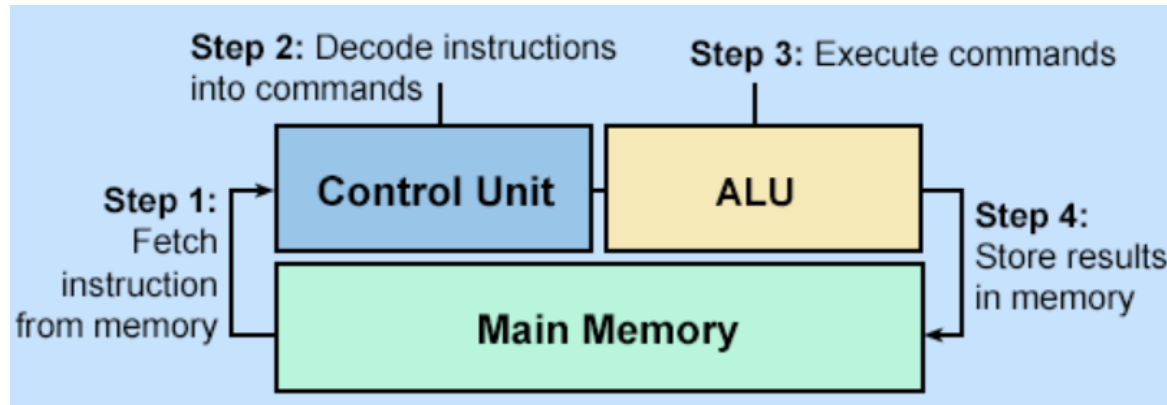
System Software and Program Execution

T. Venkatesh & John Jose



**Department of Computer Science & Engineering
Indian Institute of Technology Guwahati**

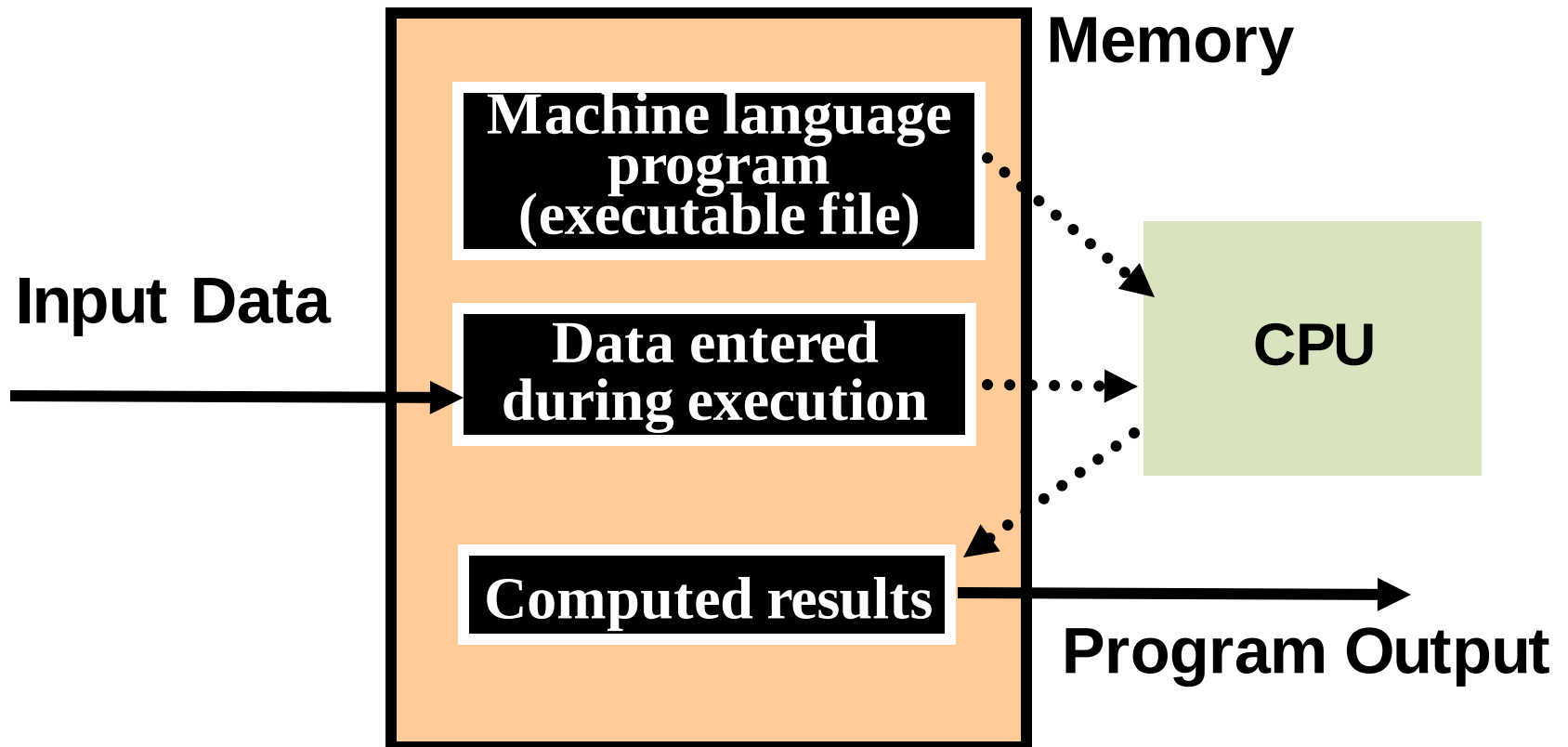
Program Execution



- CPU does the following steps to run a program (instructions are in machine language).
 1. Fetch an instruction
 2. Decode the instruction & bring data (if needed)
 3. Execute the operation
 4. Store the results (if needed)

Running Programs

- Computer goes through different steps to run a program
- The program has to be in machine language and should be in executable format in the memory to execute



Computer Languages

- Machine Language (ML)
 - Uses binary code
 - Machine-dependent
 - Not portable
- Assembly Language (AL)
 - Uses mnemonics
 - Machine-dependent
 - Not usually portable
- High-Level Language (HLL)
 - Uses English-like language
 - Machine independent
 - Portable (but must be compiled for different platforms)
 - Examples: Pascal, C, C++, Java, Fortran, . . .

Machine Language

- The representation of a computer program which can be understood by the computer.
- A program in machine code consists of a sequence of machine instructions.
- Machine instructions are in binary code
- Machine instruction has 2 parts
 - Opcode- what is the operation
 - Operand(s) – where to perform the operation

ADD R1, R2, R3 [R1 ← R2+R3]
1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 1 [16 bit instruction]

MOVE R4, R7 [R4 ← R7]

1 1 1 1 0 1 0 0 0 1 1 1 0 0 0 0

JUMPEQ R2, R6 15 [If R2==R6, PC ← PC+15]

0 0 0 1 0 0 1 0 0 1 1 0 1 1 1 1

Assembly Language

- A symbolic representation of the machine language of a specific processor.
- Is converted to machine code by an assembler.
- Usually, each line of assembly code produces one machine instruction (One-to-one correspondence).
- Mnemonic representation of the instructions and data

```
; Example of IBM PC assembly language
; Accepts a number in register AX;
; subtracts 32 if it is in the range 97-122;
; otherwise leaves it unchanged.
```

```
SUB32  PROC           ; procedure begins here
        CMP  AX,97     ; compare AX to 97
        JL   DONE      ; if less, jump to DONE
        CMP  AX,122    ; compare AX to 122
        JG   DONE      ; if greater, jump to DONE
        SUB  AX,32     ; subtract 32 from AX
DONE:   RET            ; return to main program
SUB32  ENDP           ; procedure ends here
```

High-level language

- A programming language which use statements consisting of English-like keywords such as "FOR", "PRINT" or "IF", ... etc.
- Each statement corresponds to several machine language instructions (one-to-many correspondence).
- Much easier to program than in assembly language.
- Data are referenced using descriptive names.
- Operations can be described using familiar symbols.

High level code **Address**

```
sum = 0;
for (i = 0; i < 3; i++)
{
    sum = sum + 5;
}
```

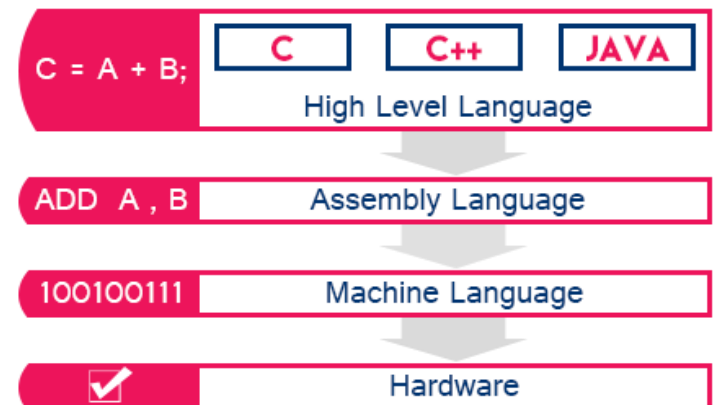
100 sum

Assume R0 will always contain the number zero **Assembly code** **Address**

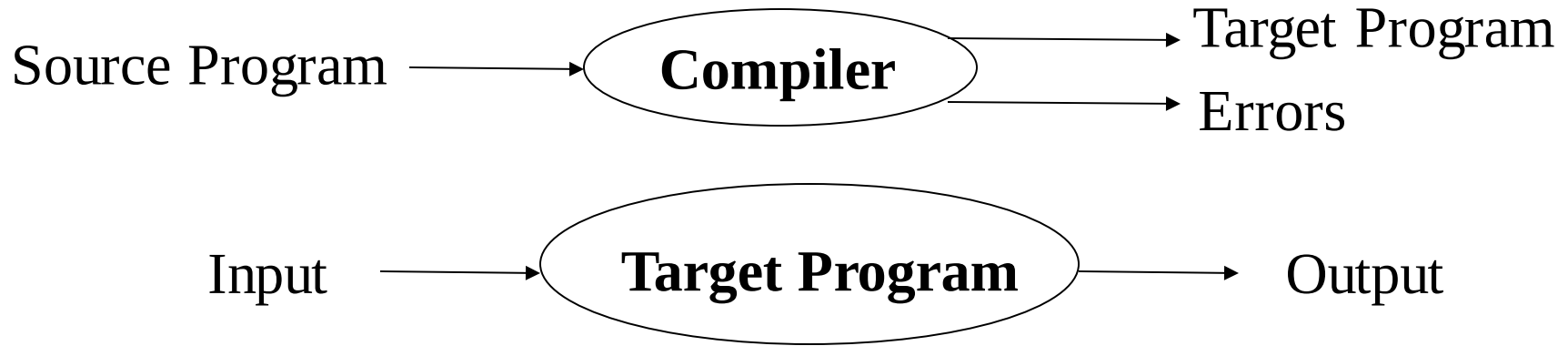
```
LUI R2, #0 ; sum = 0
LUI R3, #3 ; to set up loop
LOOP: ADDI R2, R2, #5 ; accumulate sum
      ADDI R3, R3, #-1 ; modify loop counter
      BNEZ R3, LOOP ; done looping?
      SD 100(R0), R2 ; store final result in sum
```

100 15 sum

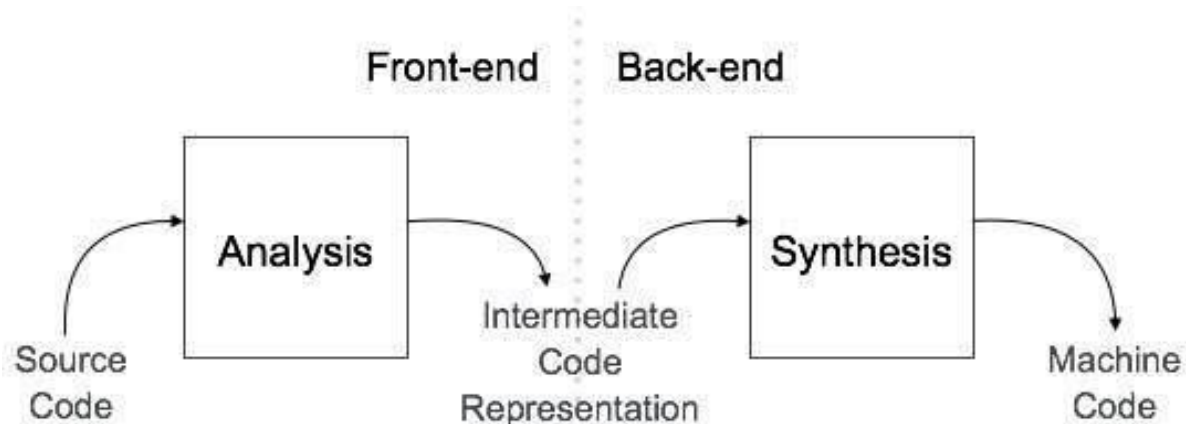
(A purple arrow points from the value 15 in the memory box to the instruction "store final result in sum")



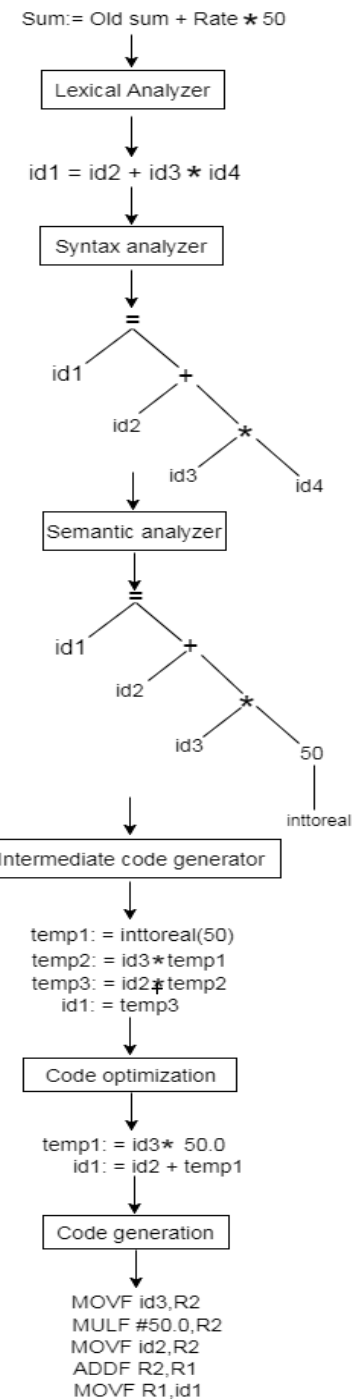
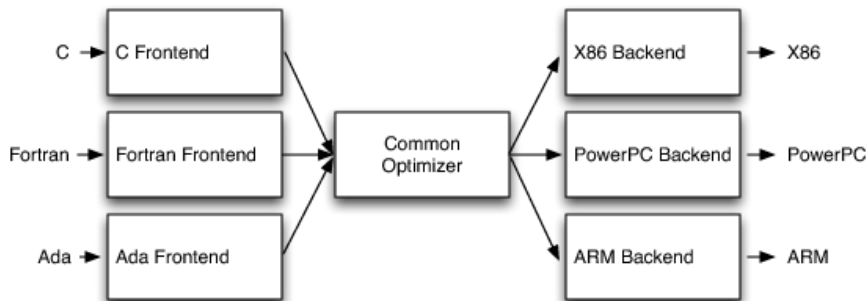
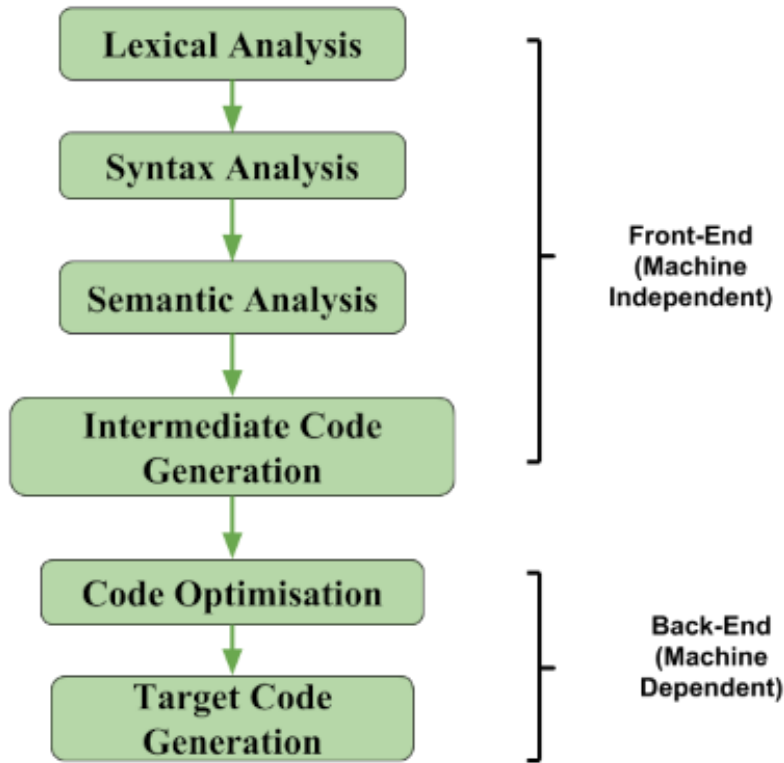
Compiler



- A compiler is a computer program that translates computer code written in High Level Language (source program) into Machine language (target program).



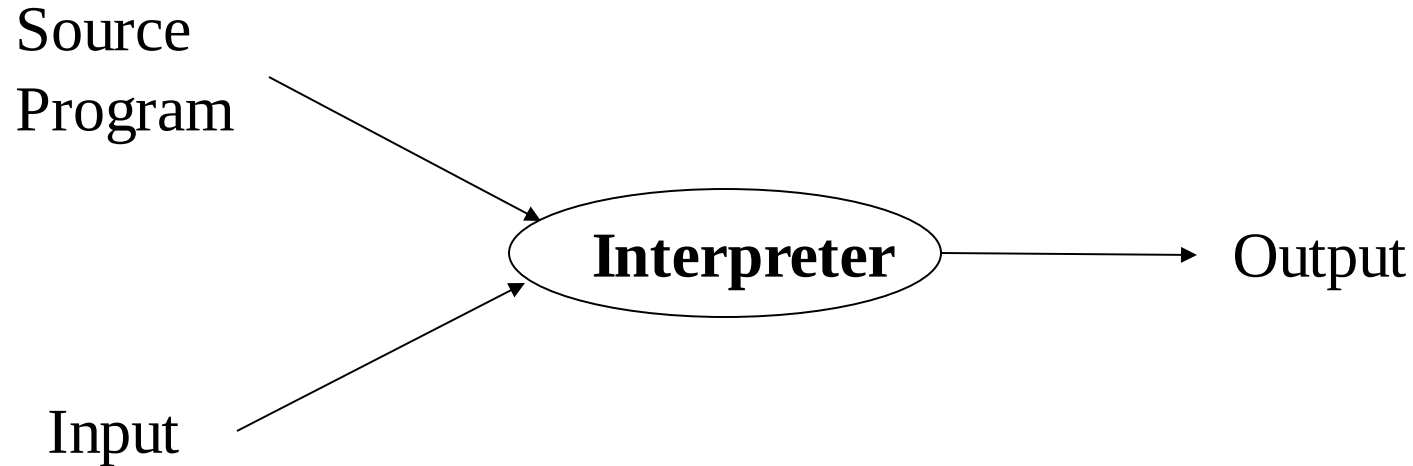
Phases of Compiler



Program Errors

- Syntax Errors:
 - Errors in grammar of the language
- Runtime error:
 - When there are no syntax errors, but the program can't complete execution
 - Divide by zero
 - Invalid input data
- Logical errors:
 - The program completes execution, but delivers incorrect results
 - Incorrect usage of parentheses
 - Wrong logic used

Interpretation



- Interpreter reads line by line, translates code line by line and executes statements one at a time
- Makes it easier to detect errors but potentially slowing down the program.

Compilation vs. Interpretation

Compiler

Compiler converts a program into machine code as a whole

Compiler creates object code file

Compiler converts high-level program that can be execution many times

Program execution is fast

Compiler displays syntax errors after compiling the whole program

Interpreter

Interpreter converts a program into machine code statement by statement

Interpreter does not create object code file

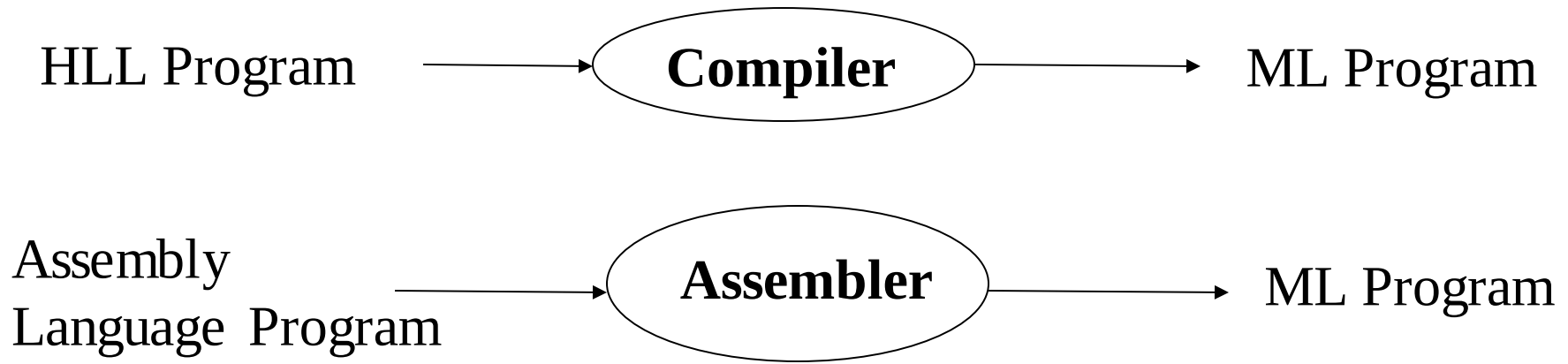
Interpreter converts high-level program each time it is execution

Program execution is slow

Interpreter displays syntax error on each statement of program

Compiler Vs. Assembler

- Assembler is a software that converts an assembly language code to machine code.
- It takes basic computer commands and converts them into binary code that computer's processor can use to perform its basic operations.



Source, Object and Executable Programs

- **Source program**

- The form in which a programmer writes a computer program in some formal programming language.
- Files usually have extension '.c', '.cpp', '.java', '.py'

- **Object program**

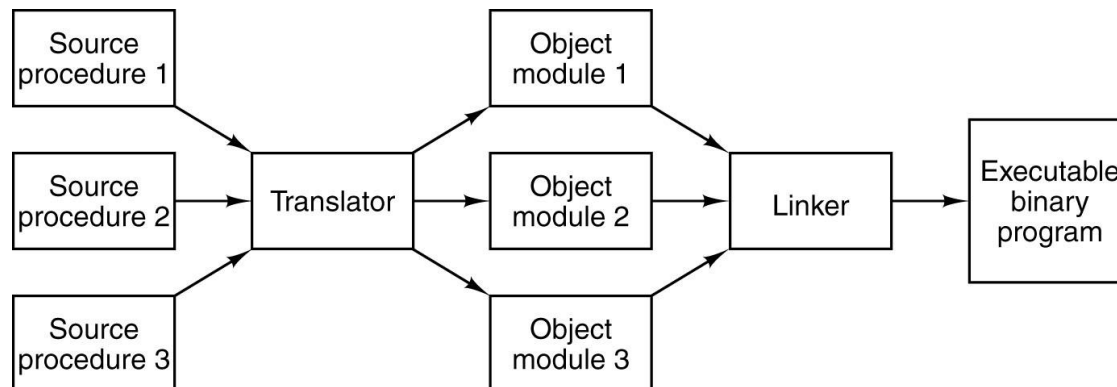
- Output from the compiler
- Equivalent machine language translation of the source program
- Files usually have extension '.obj'

- **Executable program**

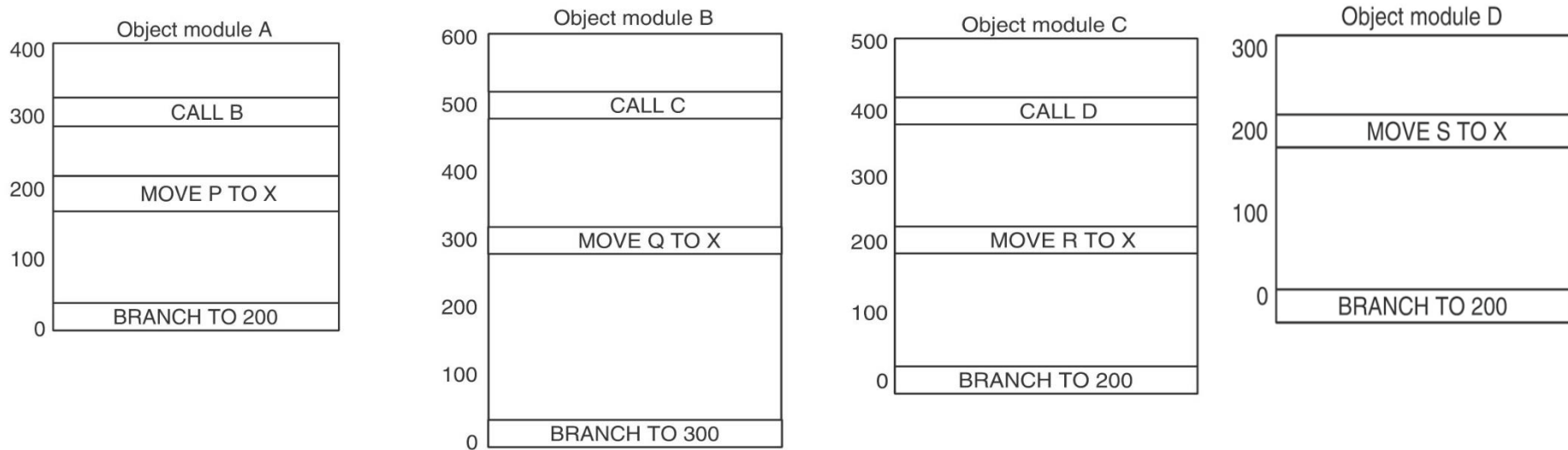
- Output from linker/loader
- Machine language program linked with necessary libraries and other files
- Files usually have extension '.exe'

Linker

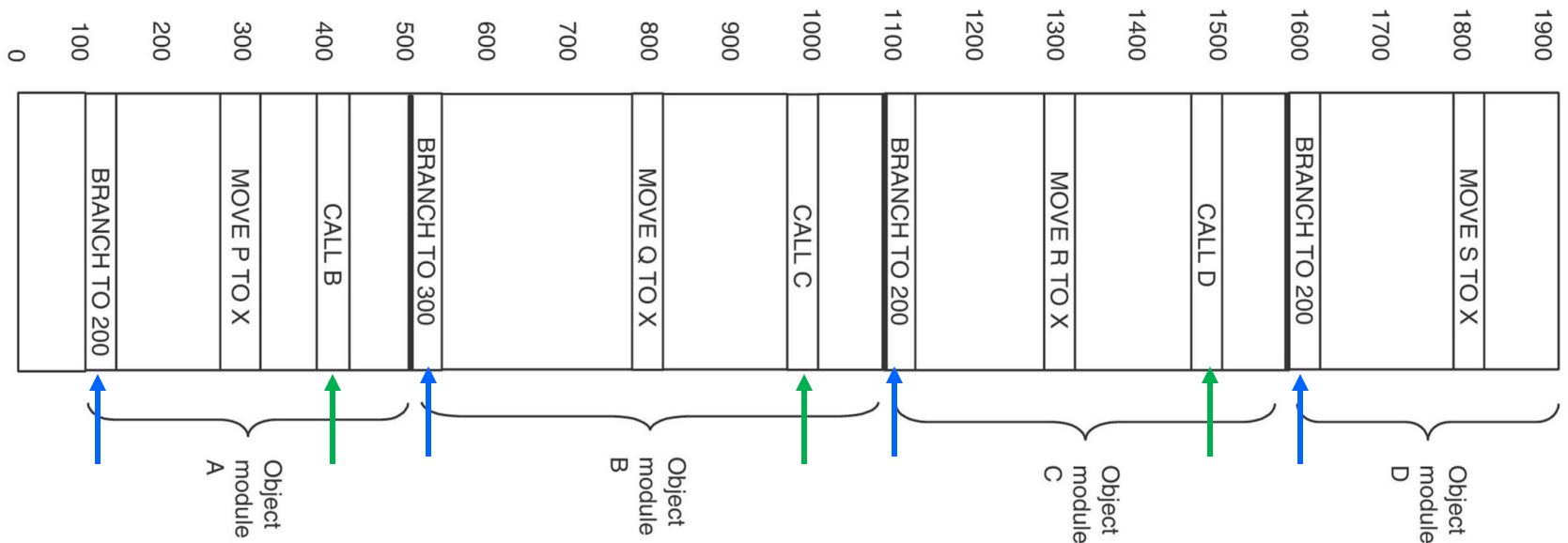
- A program that pulls other programs together so that they can run.
- Most programs are very large and consist of several modules and may need standard libraries. to run.
- The linker pulls everything together, makes sure that references to other parts of the program (code) are resolved.
- Generation of an executable binary program from a collection of independently translated source procedures requires using a linker.



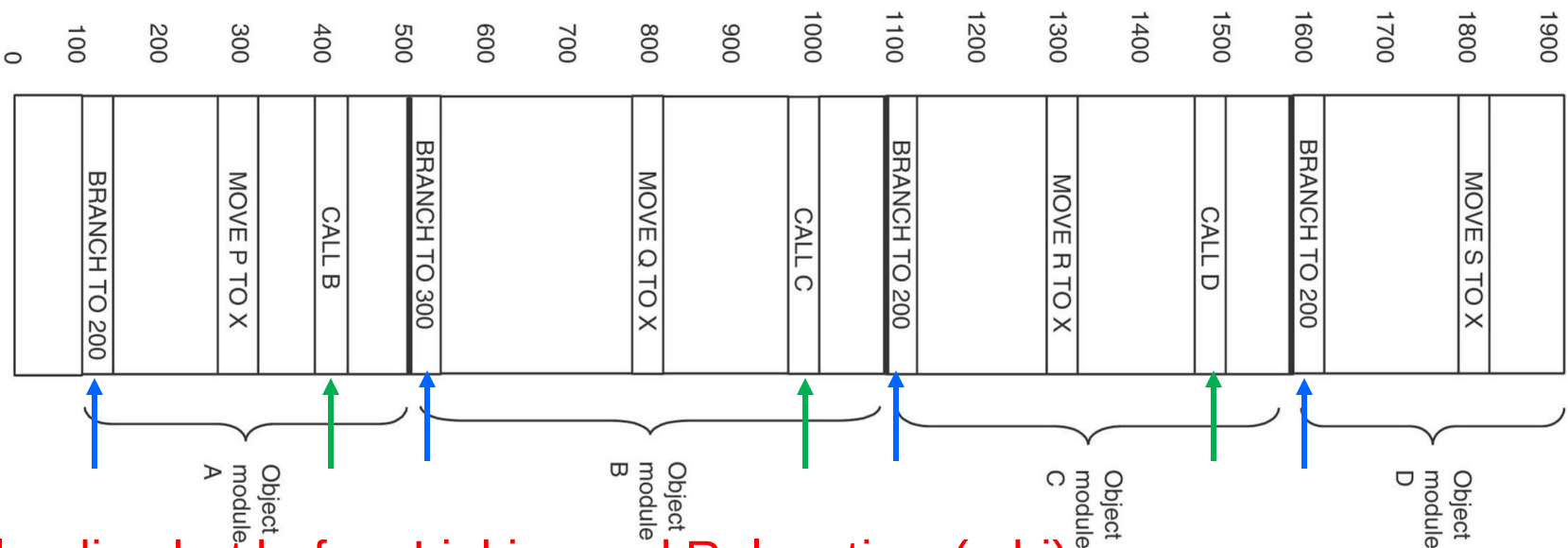
Program – A collection of object files



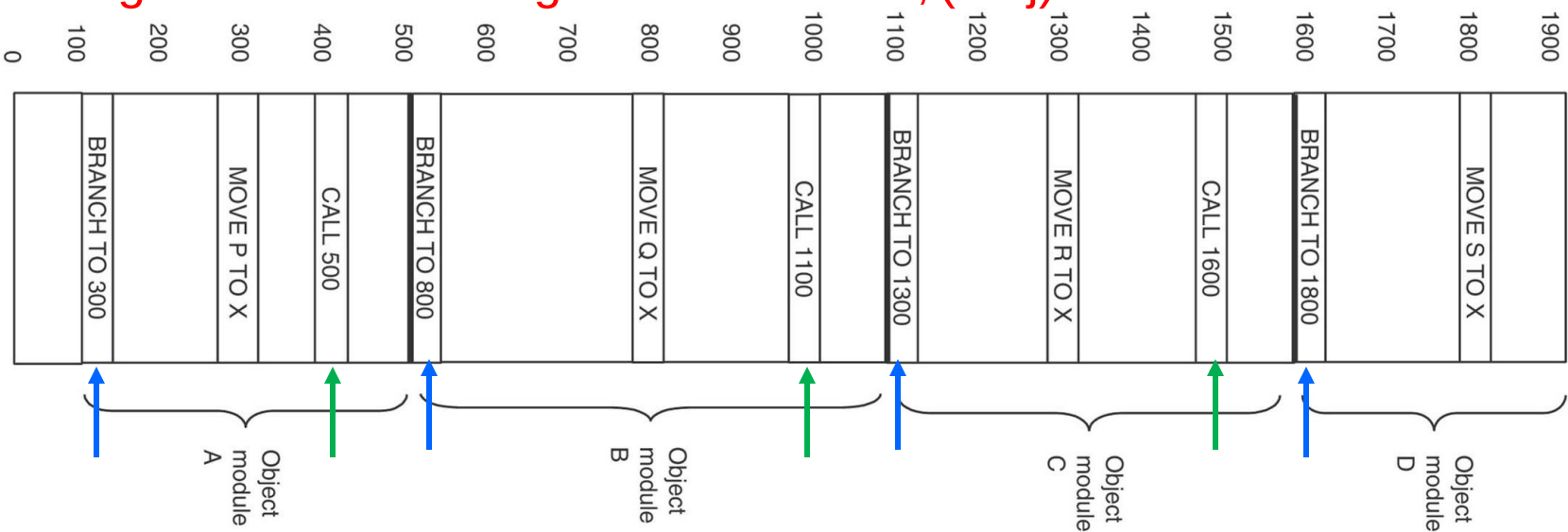
Each module has its own address space, starting at 0.



Linking and Relocation

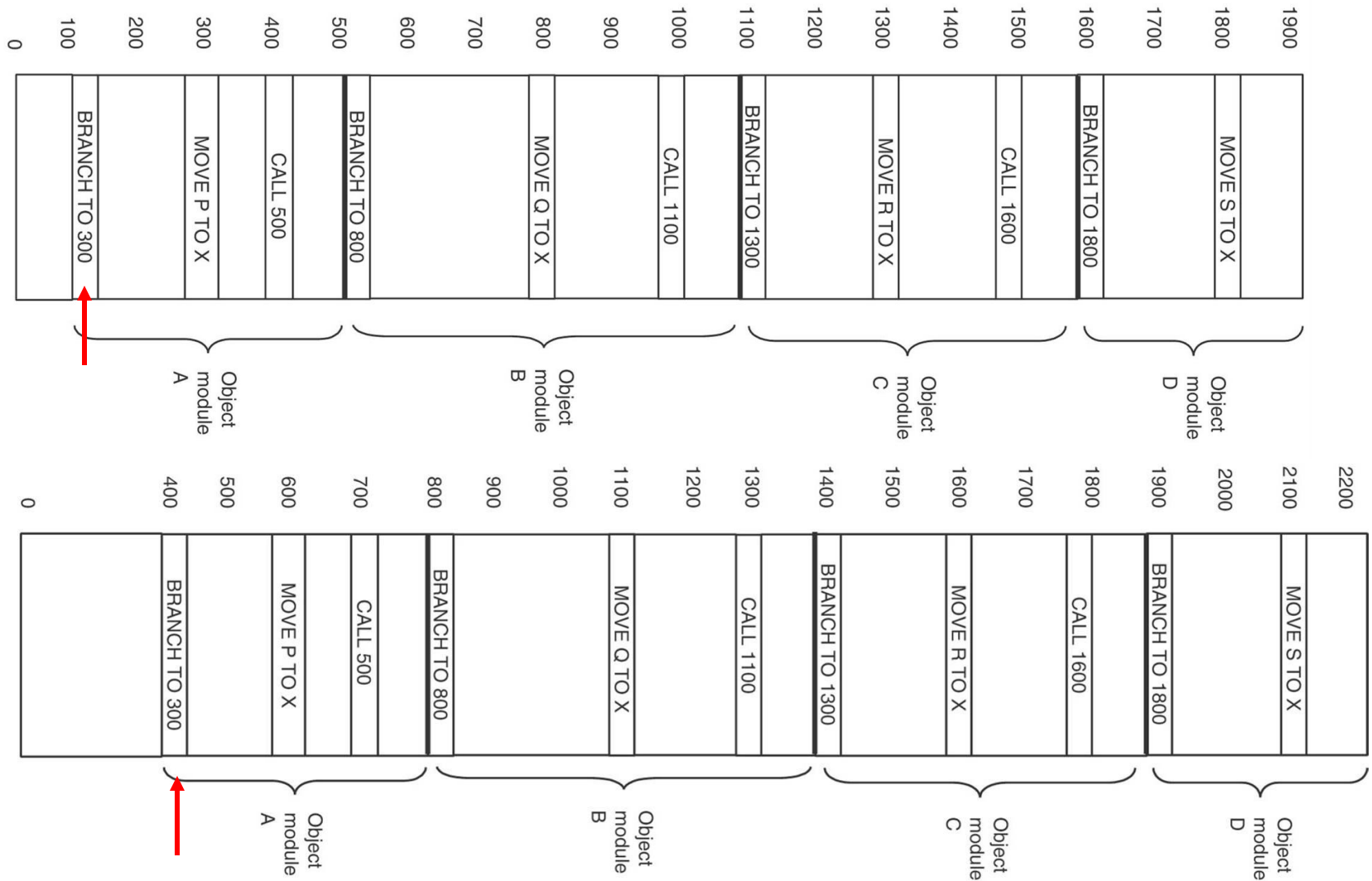


After loading but before Linking and Relocation, (.obj)



After Linking and Relocation (.exe)

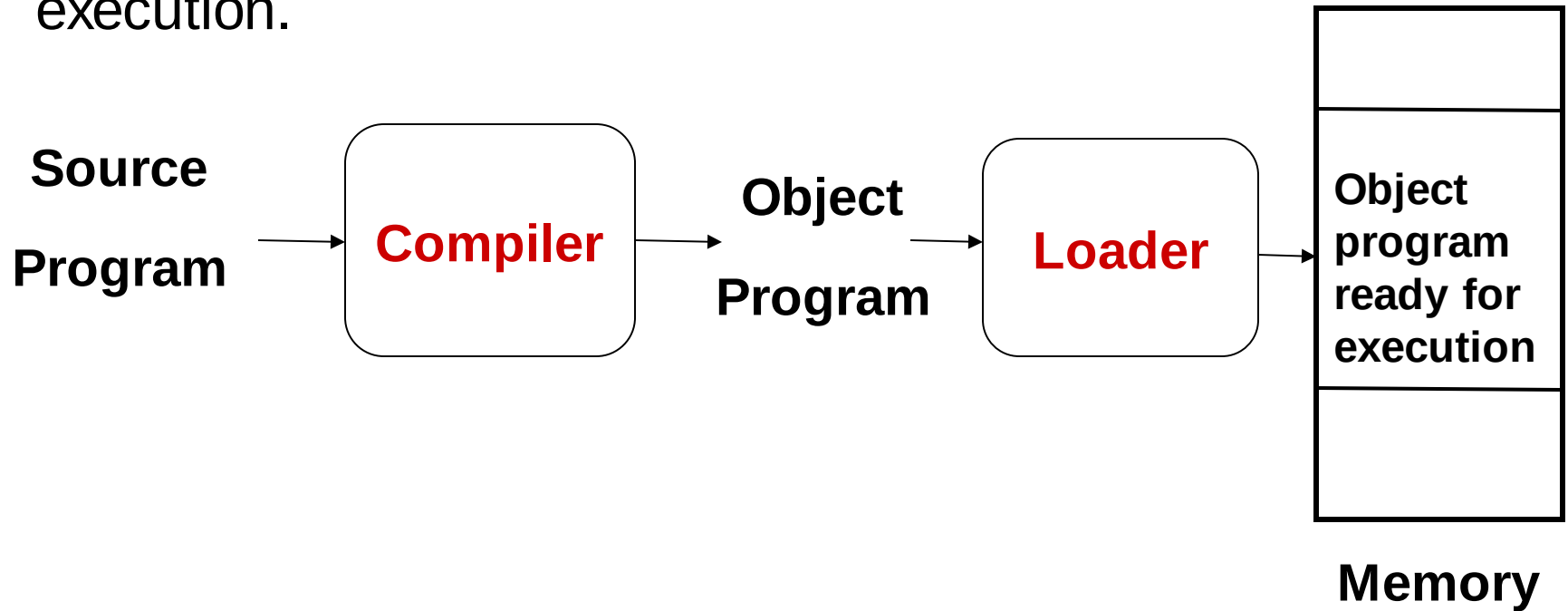
Binding Time and Dynamic Relocation



Load address changes- Need for dynamic relocation.

Loader

- Loader is a system program that copies the instructions and data from secondary memory to main memory.
- It brings object program into memory and make it ready for execution.



Functions of Loader

- **Loading:** Loads the object files into the memory of the computer system.
- **Address Binding:** Maps the virtual addresses used in the object file to physical addresses in the memory. This ensures that the program can execute correctly in the memory.
- **Linking:** Performs linking (if linker and loader are single unit), which involves resolving external references to libraries or other executable files.
- **Relocation:** Involves adjusting the program's references to memory locations based on the actual physical address.
- **Memory Management:** Manages memory by allocating and deallocating memory space for the program as needed.

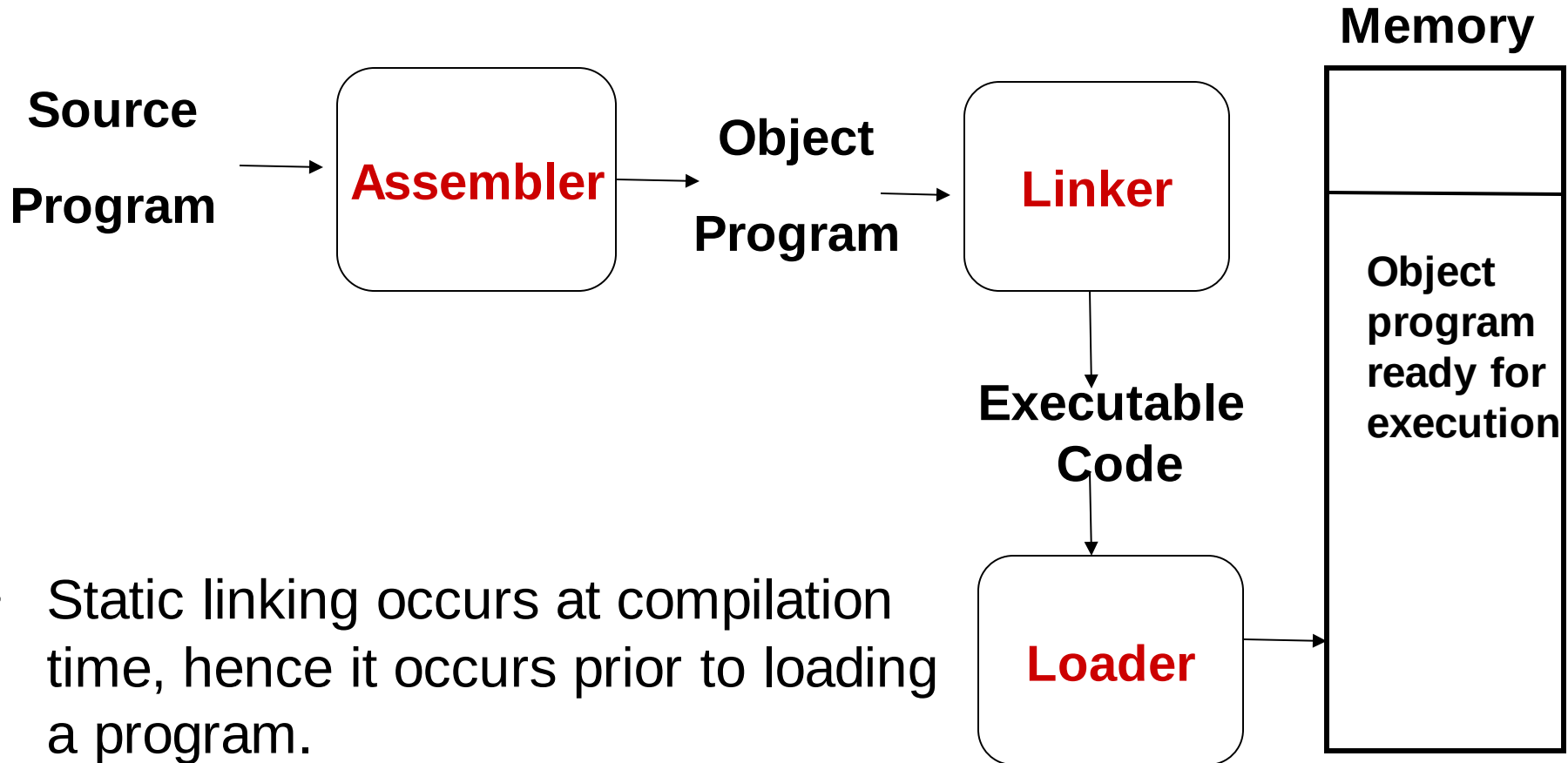
Classifications of Loader

- **Compile and go Loader:** Source code goes into the translator, and the object code is loaded into specific location in memory. Reloading needs re-compilation. Programmer should know load address before compilation.
- **Absolute Loader:** Loads the object code (output of compiler) to specified location in memory. This specific memory location is specified in the program's object code by compiler. This location is usually absolute and does not change when the program is loaded into memory.
- **Linking Loader:** Performs linking which involves resolving external references to libraries or other object files and then loads the single executable file to memory at designated address.

Classifications of Loader

- **Relocating Loader:** It loads a program at any memory location, and then adjusts all memory references in the program to reflect the new location. This allows the same program to be loaded into different memory locations without having to modify the program's object code.
- **Bootstrap Loader:** It is a computer program that is responsible for booting a computer. Loads OS kernel to main memory and handover the control to OS kernel.

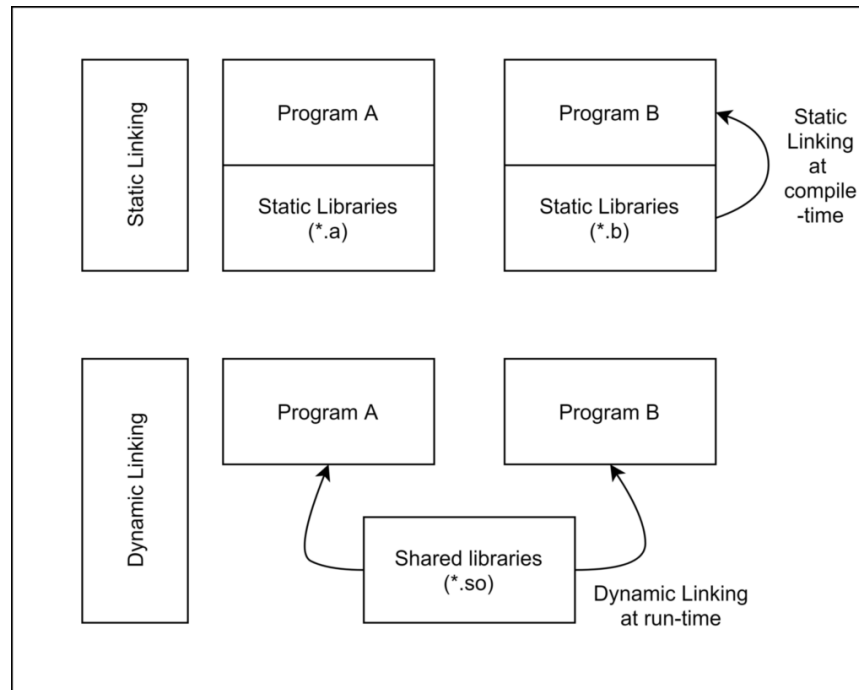
Role of Loader and Linker



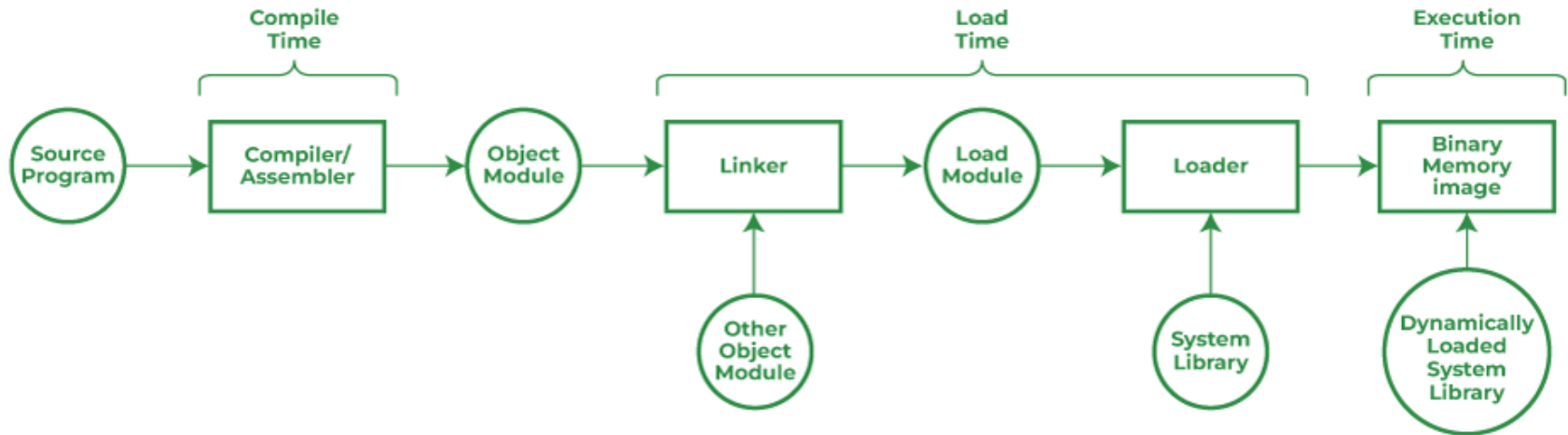
- Static linking occurs at compilation time, hence it occurs prior to loading a program.
- Dynamic linking occurs at run time, so it occurs after or at the time of the loading of a program.

Static and Dynamic Linking

- Static linking occurs at compilation time, hence it occurs prior to loading a program.
- Dynamic linking occurs at run time, so it occurs after or at the time of the loading of a program.



Program Execution and System Software



Thanks